Denoising Diffusion Probabilistic Models

From the reverse SDE to DDPM

Anton Selitskiy

Fall 2026

ESE 670 Bayesian Deep Learning Stony Brook University

Where we are

- **Sampling** (MCMC, Langevin, HMC): the target is known up to Z; we use $\nabla \log p$.
- **Generative modeling:** the target p_{data} is unknown; we only have samples (images, audio, . . .).
- **Last time:** a diffusion $\mathrm{d}X = b\,\mathrm{d}t + \sigma\,\mathrm{d}W$ can be run backwards (Anderson); the reverse drift needs only the score $\nabla \log p_t$ of the current density. Brownian bridge, toy two-point model.

Today's plan:

1. a *forward* SDE turns data into noise (known, exact);
2. its *reversal* turns noise into data, and needs only $\nabla \log p_t$;
3. discretizing the reversal gives a Gaussian Markov chain: *variance fixed*, *mean* unknown;
4. learn the mean by maximizing a variational bound $\Rightarrow$ predict the noise; the best noise predictor *is* the score;
5. what the sampler gets right, what it does not, and the VP / VE variants.

The forward process

Forward diffusion: the variance-preserving SDE

$$\boxed{\mathrm{d}x_t = -\tfrac{1}{2}\beta_t\, x_t\, \mathrm{d}t + \sqrt{\beta_t}\, \mathrm{d}W_t}, \qquad x_0 \sim p_{\mathrm{data}},$$

$\beta_t > 0$: the *noise schedule*. For $\beta \equiv 2$: Ornstein–Uhlenbeck = Langevin dynamics for $\mathcal{N}(0, I)$.

- **Densities:** $\partial_t p_t = \nabla \cdot \left(\tfrac{1}{2}\beta_t\, x\, p_t\right) + \tfrac{1}{2}\beta_t \Delta p_t$.
- **Particles:** the SDE is linear and can be solved. With $\bar{\alpha}(t) = e^{-\int_0^t \beta_s\, \mathrm{d}s}$,

$$x_t = \sqrt{\bar{\alpha}(t)}\, x_0 + \sqrt{1 - \bar{\alpha}(t)}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

- **Expectations:** $\dot{m} = -\tfrac{1}{2}\beta m$, $\dot{v} = -\beta v + \beta$ (closed: linear drift) $\Rightarrow$ mean $\sqrt{\bar{\alpha}}\, m_0$, variance $\to 1$.

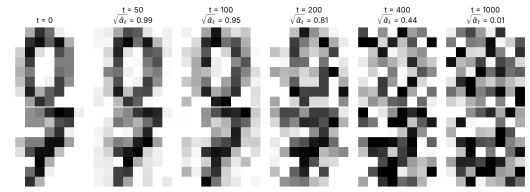
The data are shrunk while noise is added: $p_t \to \mathcal{N}(0, I)$ exponentially fast. Choose T with $\bar{\alpha}(T) \approx 0$.

The forward Markov chain

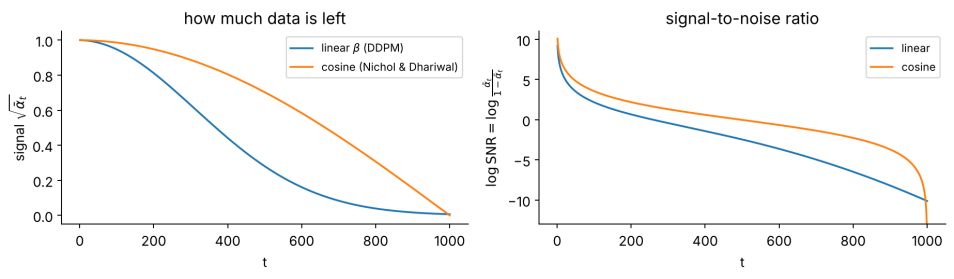
On a grid $t_0 < t_1 < \cdots < t_T$, the exact transition over one interval is

$$x_k = \sqrt{1-\tilde{\beta}_k}\, x_{k-1} + \sqrt{\tilde{\beta}_k}\, \varepsilon, \qquad q(x_k \mid x_{k-1}) = \mathcal{N}\big(\sqrt{1-\tilde{\beta}_k}\, x_{k-1},\ \tilde{\beta}_k I\big), \qquad 1-\tilde{\beta}_k = e^{-\int_{t_{k-1}}^{t_k} \beta}.$$

- **Exact, not Euler–Maruyama:** no discretization error; it keeps $\mathcal{N}(0, I)$ invariant exactly.
- Composing: $q(x_k \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_k}\, x_0, (1-\bar{\alpha}_k) I\big)$, $\bar{\alpha}_k = \prod_{j \le k}(1-\tilde{\beta}_j)$.
- **One-shot noising:** $x_k = \sqrt{\bar{\alpha}_k}\, x_0 + \sqrt{1-\bar{\alpha}_k}\, \varepsilon$ — no need to run the chain.
- DDPM: $T = 1000$, $\tilde{\beta}$ linear from 10^{-4} to $0.02 \Rightarrow \sqrt{\bar{\alpha}_T} = 0.006$.



Noise schedules



- The linear schedule destroys the signal early: the last few hundred steps are almost pure noise.
- The cosine schedule (Nichol & Dhariwal 2021) spreads the signal-to-noise ratio more evenly over the steps.
- What matters is the range of $\log \mathrm{SNR}$ covered (roughly $+10$ to -10) and how steps are spread over it.

The reverse process

Reverse diffusion

Anderson's theorem with $b = -\frac{1}{2}\beta x$, noise $\sqrt{\beta}$: in reversed time $s = T - t$,

$$\boxed{\mathrm{d}y_s = \left[\tfrac{1}{2}\beta\, y_s + \beta\, \nabla \log p_{T-s}(y_s)\right] \mathrm{d}s + \sqrt{\beta}\; \mathrm{d}\bar{W}_s, \qquad y_0 \sim p_T \approx \mathcal{N}(0, I)}$$

(derivation: write Fokker–Planck as transport, $\partial_t p = -\nabla \cdot \big((b - \frac{1}{2}\beta \nabla \log p)p\big)$, reverse time, add and subtract $\frac{1}{2}\beta \Delta p$).

- **The noise is not changed by reversal.** Only the drift changes.
- **The only unknown is the score** $s_t(x) = \nabla \log p_t(x)$ — no normalizing constant.
- **Probability-flow ODE** with the same marginals: $\dot{x} = -\frac{1}{2}\beta x - \frac{1}{2}\beta \nabla \log p_t(x)$ (run backwards).

If we knew the score at every noise level, simulating the reverse SDE from $\mathcal{N}(0, I)$ would give exact samples of p_{data}.

Discretizing the reverse SDE

Euler–Maruyama from t to $t - \Delta t$:

$$x_{t-\Delta t} = x_t + \big(\tfrac{1}{2}\beta_t x_t + \beta_t \nabla \log p_t(x_t)\big)\Delta t + \sqrt{\beta_t \Delta t}\,\varepsilon.$$

Each reverse step is Gaussian:

- its **mean** depends on the unknown score $\Rightarrow$ learn it;
- its **variance** $\beta_t \Delta t \approx \tilde{\beta}_t$ is fixed by the noise $\Rightarrow$ nothing to learn (to leading order).

The model

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\ \sigma_t^2 I\big), \qquad p_\theta(x_{0:T}) = \mathcal{N}(x_T; 0, I) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t).$$

$\sigma_t^2 = \tilde{\beta}_t$ or $\hat{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\tilde{\beta}_t$: they differ only by $O(\Delta t^2)$ and only near $t = 0$.

Why Gaussian only for small steps

The true reverse kernel, by conditioning on the clean data x_0:

$$q(x_{t-1} \mid x_t) = \mathbb{E}_{x_0 \sim q(x_0 \mid x_t)} \Big[\mathcal{N}\big(x_{t-1};\ \hat{\mu}_t(x_t, x_0),\ \hat{\beta}_t I\big) \Big] \qquad \text{— a } \textbf{mixture} \text{ of Gaussians.}$$

- The forward drift is linear $\Rightarrow$ forward kernel Gaussian for every step. The reverse drift contains the score (nonlinear) $\Rightarrow$ the reverse kernel is Gaussian only over short steps.
- One large step with two data points: $q(x_0 \mid x_1)$ sits on the two points — not Gaussian at all.
- Small steps: the means of the mixture differ by $O(\tilde{\beta})$, the widths are $O(\sqrt{\tilde{\beta}})$ $\Rightarrow$ they overlap $\Rightarrow$ nearly Gaussian (Feller 1949; Sohl-Dickstein et al. 2015).

This is why DDPM uses $T \approx 1000$ steps. Few steps need better ODE solvers or non-Gaussian reverse kernels.

Training: the variational bound

The evidence lower bound

Maximum likelihood: $-\mathbb{E}_{p_{\text{data}}} \log p_\theta(x_0) \to \min$, with $p_\theta(x_0) = \int p_\theta(x_{0:T})\, \mathrm{d}x_{1:T}$ intractable.

Since $p_\theta(x_0) = p_\theta(x_{0:T}) / p_\theta(x_{1:T} \mid x_0)$ for every path $x_{1:T}$, average over the forward path $q(x_{1:T} \mid x_0)$:

$$
\begin{aligned}
\log p_\theta(x_0) &= \int q(x_{1:T} \mid x_0)\, \log p_\theta(x_0)\, \mathrm{d}x_{1:T} \\
&= \int q(x_{1:T} \mid x_0)\, \log \Big(\frac{p_\theta(x_{0:T})}{p_\theta(x_{1:T} \mid x_0)}\, \frac{q(x_{1:T} \mid x_0)}{q(x_{1:T} \mid x_0)} \Big)\, \mathrm{d}x_{1:T} \\
&= \underbrace{\int q(x_{1:T} \mid x_0) \log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\, \mathrm{d}x_{1:T}}_{=:\, \mathcal{L}_\theta} + \underbrace{\int q(x_{1:T} \mid x_0) \log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{1:T} \mid x_0)}\, \mathrm{d}x_{1:T}}_{= \mathrm{KL}(q(x_{1:T} \mid x_0) \,\|\, p_\theta(x_{1:T} \mid x_0)) \,\geq\, 0} \\
&\geq \mathcal{L}_\theta = \mathbb{E}_{q(x_{1:T} \mid x_0)} \Big[\log \frac{p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t)}{\prod_{t=1}^T q(x_t \mid x_{t-1})} \Big].
\end{aligned}
$$

- The gap $\log p_\theta(x_0) - \mathcal{L}_\theta$ is exactly the KL between the forward (noising) path and the model's reverse path, given x_0.
- Averaging over the data: $-\mathbb{E}_{p_{\text{data}}} \log p_\theta(x_0) \leq -\mathbb{E}_{p_{\text{data}}} \mathcal{L}_\theta \to \min$.

Decomposition into KL terms

For $t \geq 2$, by the Markov property and Bayes' rule,

$$q(x_t \mid x_{t-1}) = q(x_t \mid x_{t-1}, x_0) = \frac{q(x_{t-1} \mid x_t, x_0)\, q(x_t \mid x_0)}{q(x_{t-1} \mid x_0)}.$$

Substituting into $\mathcal{L}_\theta$, the ratios $q(x_{t-1} \mid x_0)/q(x_t \mid x_0)$ telescope to $q(x_1 \mid x_0)/q(x_T \mid x_0)$, and $q(x_1 \mid x_0)$ cancels with the last term. Integrating out the variables that do not appear in each term:

$$\mathcal{L}_\theta = \underbrace{-\,\mathrm{KL}\left(q(x_T \mid x_0) \,\|\, p(x_T)\right)}_{\text{no } \theta,\ \approx 0} + \underbrace{\mathbb{E}_{q(x_1 \mid x_0)} \log p_\theta(x_0 \mid x_1)}_{\text{reconstruction / dequantization}}$$

$$- \sum_{t=2}^{T} \mathbb{E}_{q(x_t \mid x_0)} \underbrace{\mathrm{KL}\left(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\right)}_{\text{one reverse step vs the forward posterior}}.$$

The middle terms do the work: each compares one step of the model with the *forward posterior* $q(x_{t-1} \mid x_t, x_0)$, the reverse step one would take if the clean data x_0 were known.

Reconstruction term. The last reverse step is the "decoder" $x_1 \mapsto x_0$ (as in a VAE). Pixels are discrete (0..255), so DDPM makes it a *discrete* decoder: $p_\theta(x_0 \mid x_1) = \prod_i \int_{x_0^i - 1/255}^{x_0^i + 1/255} \mathcal{N}\left(x; \mu_\theta^i(x_1, 1), \sigma_1^2\right) \mathrm{d}x$ (outer bins to $\pm\infty$), so the bound is a true discrete log-likelihood (bits/dim). Alternative: dequantize the data by adding uniform noise. In $\mathcal{L}_{\mathrm{simple}}$ the $t = 1$ term of the ε-loss stands in for it; sampling ends with $z = 0$.

The forward posterior: Gaussian only given x_0

Given x_0, the forward chain is linear with Gaussian noise: $(x_0, x_1, \ldots, x_T) \mid x_0$ is **jointly Gaussian** (a Gauss–Markov process). Hence every conditional is Gaussian; by Bayes, a product of Gaussians in x_{t-1}:

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} = \mathcal{N}\big(x_{t-1};\ \hat{\mu}_t(x_t, x_0),\ \hat{\beta}_t I\big),$$

$$\hat{\mu}_t = \underbrace{\frac{\sqrt{\bar{\alpha}_{t-1}}\,\tilde{\beta}_t}{1-\bar{\alpha}_t}}_{c_0(t)}\, x_0 + \underbrace{\frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}_{c_t(t)}\, x_t, \qquad \hat{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\tilde{\beta}_t.$$

But the process itself is not Gaussian

- The marginals $q(x_t) = \int q(x_t \mid x_0)\, p_{\text{data}}(x_0)\, \mathrm{d}x_0$ are mixtures over the data: not Gaussian (except $q(x_T) \approx \mathcal{N}(0, I)$).
- The true reverse step $q(x_{t-1} \mid x_t) = \mathbb{E}_{x_0 \mid x_t}\big[q(x_{t-1} \mid x_t, x_0)\big]$ is a mixture of Gaussians.

Conditioning on the data point is what makes each KL term of the ELBO a closed-form Gaussian KL.

Three equivalent losses: mean, data, noise

Fixed variance $\sigma_t^2 \Rightarrow \mathrm{KL}\left(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\right) = \frac{1}{2\sigma_t^2}\|\hat{\mu}_t - \mu_\theta\|^2 + C_t$. Since $\hat{\mu}_t$ is linear in x_0, parametrize the model the same way:

- **data prediction:** $\mu_\theta = c_0(t)\, x_\theta(x_t, t) + c_t(t)\, x_t \quad \Rightarrow \quad \mathrm{KL} = \dfrac{c_0(t)^2}{2\sigma_t^2}\left\|x_0 - x_\theta(x_t, t)\right\|^2 + C_t;$
- **noise prediction:** $x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon}{\sqrt{\bar{\alpha}_t}}$, $x_\theta = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta}{\sqrt{\bar{\alpha}_t}} \Rightarrow \|x_0 - x_\theta\|^2 = \frac{1-\bar{\alpha}_t}{\bar{\alpha}_t}\|\varepsilon - \varepsilon_\theta\|^2$, $\mu_\theta = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\tilde{\beta}_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta\right)$,

$$\mathrm{KL} = \frac{\tilde{\beta}_t^2}{2\sigma_t^2\,\alpha_t(1-\bar{\alpha}_t)}\left\|\varepsilon - \varepsilon_\theta(x_t, t)\right\|^2 + C_t.$$

Same minimizer, $x_\theta^* = \mathbb{E}[x_0 \mid x_t]$, $\varepsilon_\theta^* = \mathbb{E}[\varepsilon \mid x_t]$; only the weight over t differs. Dropping it (Ho et al. 2020):

$$\boxed{\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\left\|\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon,\, t)\right\|^2 = \mathbb{E}\left[\mathrm{SNR}_t\,\|x_0 - x_\theta\|^2\right]}, \qquad \mathrm{SNR}_t = \tfrac{\bar{\alpha}_t}{1-\bar{\alpha}_t}.$$

Train a denoiser: predict the added noise (or the clean data) from a noised data point.

Score Function

$$s_\theta(x, t) \approx \frac{\partial}{\partial x} \log p_t(x)$$

$$\theta = \underset{\theta}{\operatorname{argmin}} \sum_{t=1}^{T-1} \int p_t(x) \left\| s_\theta(x, t) - \frac{\partial}{\partial x} \log p_t(x) \right\|^2 dx.$$

Score Function

$$\int p_t(x) \left\| s_\theta(x,t) - \frac{\partial}{\partial x} \log p_t(x) \right\|^2 dx = \int p_t(x) \left[s_\theta^T s_\theta - 2 s_\theta^T \frac{\partial}{\partial x} \log p_t(x) \right] dx + const$$

$$= \int p_t(x) s_\theta^T s_\theta \, dx - 2 \int p_t(x) s_\theta^T \frac{\partial p_t(x)}{\partial x} \frac{1}{p_t(x)} dx + const$$

$$\overset{p_t(x) = \int q_t(x|x_0) p_0(x_0) dx_0}{=} \int \int q_t(x|x_0) p_0(x_0) s_\theta^T s_\theta \, dx dx_0 - 2 \int s_\theta^T \frac{\partial}{\partial x} \int q_t(x|x_0) p_0(x_0) dx_0 dx + const$$

$$= \iint q_t(x|x_0) p_0(x_0) s_\theta^T s_\theta \, dx dx_0 - 2 \iint s_\theta^T \frac{\partial}{\partial x} q_t(x|x_0) p_0(x_0) dx_0 dx + const$$

$$= \iint q_t(x|x_0) p_0(x_0) s_\theta^T s_\theta \, dx dx_0 - 2 \iint s_\theta^T q_t(x|x_0) \frac{\partial}{\partial x} \log q_t(x|x_0) p_0(x_0) dx_0 dx + const$$

$$= \iint q_t(x|x_0) p_0(x_0) \left[s_\theta^T s_\theta - 2 s_\theta^T \frac{\partial}{\partial x} \log q_t(x|x_0) \pm \left(\tfrac{\partial}{\partial x} \log q_t(x|x_0) \right)^T \tfrac{\partial}{\partial x} \log q_t(x|x_0) \right] dx_0 dx + const$$

$$= \iint q_t(x|x_0) p_0(x_0) \left\| s_\theta - \frac{\partial}{\partial x} \log q_t(x|x_0) \right\|^2 dx_0 dx + const$$

Noise prediction is score estimation

Conditional score of $q(x_t \mid x_0)$: $\nabla_{x_t} \log q(x_t \mid x_0) = -\dfrac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}$.

Marginal score: differentiate $p_t(x_t) = \int q(x_t \mid x_0) p_{\text{data}}(x_0)\,\mathrm{d}x_0$ under the integral:

$$\nabla \log p_t(x_t) = \mathbb{E}\big[\nabla_{x_t} \log q(x_t \mid x_0) \;\big|\; x_t\big] = -\frac{\mathbb{E}[\varepsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}}.$$

The regression target: the minimizer of $\mathcal{L}_{\text{simple}}$ over *all* functions is the conditional mean $\varepsilon^*(x_t, t) = \mathbb{E}[\varepsilon \mid x_t]$. So

$$\boxed{\nabla \log p_t(x_t) = -\frac{\varepsilon^*(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}} \qquad \text{(denoising score matching; Vincent 2011)}$$

The trained network only approximates it, $\varepsilon_\theta \approx \varepsilon^*$, hence $s_\theta := -\varepsilon_\theta/\sqrt{1-\bar{\alpha}_t} \approx \nabla \log p_t$.

Tweedie: $\mathbb{E}[x_0 \mid x_t] = \dfrac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon^*(x_t, t)}{\sqrt{\bar{\alpha}_t}} \;\approx\; \dfrac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} =: \hat{x}_0(x_t)$.

$*$ = ideal minimizer (exact identities); no star = trained network (approximations).

One network, four readings: noise ε_θ $\Leftrightarrow$ score s_θ $\Leftrightarrow$ denoiser $\hat{x}_0$ $\Leftrightarrow$ reverse mean μ_θ.

Sampling

DDPM sampling is Euler–Maruyama for the reverse SDE

With $\varepsilon_\theta = -\sqrt{1-\bar{\alpha}_t}\, s_t$: $\mu_\theta = \dfrac{x_t + \tilde{\beta}_t\, s_t(x_t)}{\sqrt{\alpha_t}} = x_t + \big(\tfrac{1}{2}\beta_t x_t + \beta_t s_t(x_t)\big)\Delta t + O(\Delta t^2)$: the variational bound and Anderson's SDE give the **same chain**.

Algorithm 1 DDPM (Ho et al. 2020)

Training: repeat: $x_0 \sim$ data, $t \sim U\{1..T\}$, $\varepsilon \sim \mathcal{N}(0, I)$; gradient step on $\|\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, t)\|^2$

Sampling:

1: $x_T \sim \mathcal{N}(0, I)$

2: **for** $t = T, \ldots, 1$ **do**

3: $\quad z \sim \mathcal{N}(0, I)$ if $t > 1$, else $z = 0$

4: $\quad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\big(x_t - \frac{\tilde{\beta}_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\big) + \sigma_t z$

One reverse step = transport + Langevin sampling

Split the score term of the reverse SDE in half (reversed time $s = T - t$):

$$\mathrm{d}y = \underbrace{\left[\tfrac{1}{2}\beta y + \tfrac{1}{2}\beta \nabla \log p_t(y)\right] \mathrm{d}s}_{\text{probability-flow ODE: moves } p_t \to p_{t-\mathrm{d}s}} + \underbrace{\tfrac{1}{2}\beta \nabla \log p_t(y)\,\mathrm{d}s + \sqrt{\beta}\;\mathrm{d}\bar{W}}_{\text{Langevin dynamics for } p_t \text{ at speed } c=\beta/2}$$

Langevin for a target q at speed c is $c\,\nabla \log q\,\mathrm{d}s + \sqrt{2c}\;\mathrm{d}W$; with $c = \beta/2$ the noise is $\sqrt{\beta}$ — it matches. The Langevin part leaves p_t **invariant**: it does not move the distribution, it keeps the particles distributed as p_t.

Discretely, with $1/\sqrt{\alpha_t} \approx 1 + \frac{1}{2}\tilde{\beta}_t$ and $s_t = \nabla \log p_t$:

$$x_{t-1} = \frac{x_t + \tilde{\beta}_t s_t(x_t)}{\sqrt{\alpha_t}} + \sqrt{\tilde{\beta}_t}\, z \approx \underbrace{x_t + \tfrac{1}{2}\tilde{\beta}_t\big(x_t + s_t(x_t)\big)}_{\text{Euler step of the ODE}} + \underbrace{\tfrac{1}{2}\tilde{\beta}_t\, s_t(x_t) + \sqrt{\tilde{\beta}_t}\, z}_{\text{one ULA step, } h=\tilde{\beta}_t/2,\ \text{target } p_t} .$$

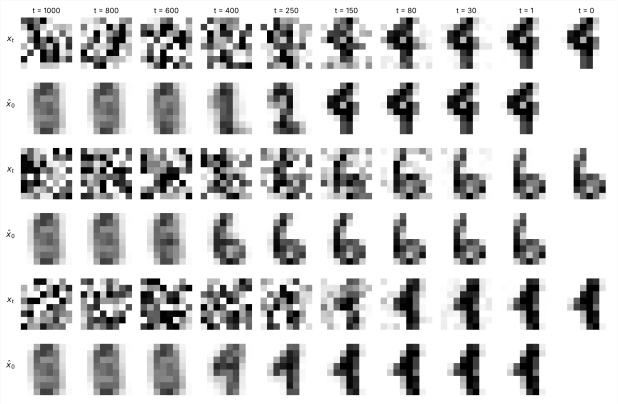
Each DDPM step = move the distribution one notch + one unadjusted Langevin step at the current noise level.

Why one Langevin step per level is enough

- **Warm start.** Before the step the particles are already $\approx p_t$, and consecutive levels are close, $\mathrm{KL}(p_{t-1} \| p_t) = O(\Delta t^2)$. The Langevin step starts in its own equilibrium: no burn-in. It only removes the small mismatch left by the transport step and by score errors.
- **Easy targets first.** At high noise $p_t \approx \mathcal{N}(0, I)$: log-concave, fast mixing. Barriers between modes appear only at low noise, when the particles have already chosen their basins (coarse to fine).
- **Error correction.** The Langevin part pulls the particles toward the current p_t: $\frac{\mathrm{d}}{\mathrm{d}s}\mathrm{KL}(\rho \| p_t)$ gets the term $-$(Fisher information) from the Langevin lecture. The ODE part only carries errors along. Hence SDE samplers (DDPM) are more robust to score errors; ODE samplers (DDIM) need fewer steps.

sampler	what it does at each noise level
probability-flow ODE (DDIM)	transport only
annealed Langevin (Song & Ermon 2019)	Langevin only, many steps per level
DDPM / reverse SDE	transport + one Langevin step
predictor–corrector (Song et al. 2021)	transport + extra Langevin steps

Watching the sampler: x_t and the denoiser's guess $\hat{x}_0(x_t)$

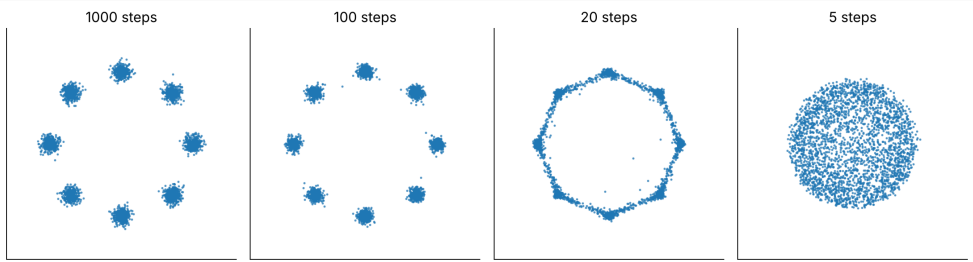


8×8 digits, DDPM schedule, exact denoiser of the training set. Rows x_t: the sample; rows $\hat{x}_0$: Tweedie's guess. At high noise the guess is a blurry *average* digit; it commits to one digit long before the noise is gone.

Two lessons from the picture

- **Coarse to fine.** The guess $\hat{x}_0$ is the posterior mean $\mathbb{E}[x_0 \mid x_t]$: an average over all images compatible with x_t. At high noise many images are compatible $\Rightarrow$ blurry average; the reverse process decides the global shape early and the details late (same as the two-bump toy: the particle chooses its mode in the middle).
- **Memorization.** With the *exact* score of the training set, every generated sample is a training image (distance to the nearest one: 0.000). The exact empirical score reproduces the data. A trained network generalizes *because* it cannot represent this score exactly: architecture, finite training, and the smoothness of ε_θ act as regularizers. Large models do sometimes memorize (Carlini et al. 2023).

How many steps?



8 Gaussians, exact score, DDPM ancestral sampling with the same continuous schedule on coarser grids. With few steps the Gaussian reverse kernel is wrong (it should be a mixture) and Euler–Maruyama is inaccurate: the modes blur. Making sampling fast is the subject of the next lectures.

What is corrected, and what is not

In MALA and HMC a Metropolis step removes the discretization error *exactly*. Not here:

	MALA / HMC	DDPM reverse chain
target density	known up to Z	p_t unknown; only $\nabla \log p_t$ (learned)
dynamics	homogeneous, invariant π	inhomogeneous, finite horizon
discretization error	removed by MH	bounded, not removed

$$\mathrm{KL}(p_0 \,\|\, \hat{p}_0) \lesssim \underbrace{\mathrm{KL}\left(p_T \,\|\, \mathcal{N}(0, I)\right)}_{\text{start}} + \underbrace{\textstyle\sum_t \Delta t\, \mathbb{E}\|s_\theta - \nabla \log p_t\|^2}_{\text{score error}} + \underbrace{O(\Delta t)}_{\text{step size}} \qquad \text{(Chen et al. 2023)}$$

Remedies: better integrators for the probability-flow ODE (DDIM, Heun, DPM-Solver) — the role of leapfrog; Langevin correctors at each noise level (predictor–corrector) — the role of the MH step.

Variants

Variance preserving vs variance exploding

One Gaussian family: $x_t = \alpha_t\, x_0 + \sigma_t\, \varepsilon$, generated by $\mathrm{d}x = f(t)\, x\, \mathrm{d}t + g(t)\, \mathrm{d}W$,
$f = \frac{\mathrm{d}}{\mathrm{d}t} \log \alpha_t$, $g^2 = \frac{\mathrm{d}\sigma_t^2}{\mathrm{d}t} - 2f\sigma_t^2$.

VP (DDPM): $\alpha_t^2 + \sigma_t^2 = 1$

$$\mathrm{d}x = -\tfrac{1}{2}\beta_t x\, \mathrm{d}t + \sqrt{\beta_t}\, \mathrm{d}W$$

$$x_k = \sqrt{1 - \tilde{\beta}_k}\, x_{k-1} + \sqrt{\tilde{\beta}_k}\, z_k$$

Shrinks the data while adding noise: forgets x_0, $p_T \approx \mathcal{N}(0, I)$.

VE (NCSN / score SDE): $\alpha_t = 1$

$$\mathrm{d}x = \sqrt{\tfrac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}}\, \mathrm{d}W, \qquad x_t = x_0 + \sigma(t)\varepsilon$$

$$x_i = x_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2}\, z_i$$

Brownian motion with a clock: never forgets $\mathbb{E}x_0$, so $\sigma_{\max} \gg$ data scale.

Reverse SDE / probability-flow ODE (whole family):

$$\mathrm{d}x = \left[f x - g^2 \nabla \log p_t\right] \mathrm{d}t + g\, \mathrm{d}\bar{W}, \qquad \dot{x} = f x - \tfrac{1}{2} g^2 \nabla \log p_t.$$

VE with $\sigma(t) = t$: $\dfrac{\mathrm{d}x}{\mathrm{d}\sigma} = \dfrac{x - D(x;\sigma)}{\sigma}$, $D(x;\sigma) = \mathbb{E}[x_0 \mid x]$ (Tweedie: $\mathbb{E}[x_0 \mid x_t] = (x_t + \sigma_t^2 \nabla \log p_t)/\alpha_t$).

Same model, rescaled; what is used in practice

Rescaling: $\dfrac{x_t}{\alpha_t} = x_0 + \dfrac{\sigma_t}{\alpha_t}\,\varepsilon$: every member is VE in x/α_t with $\tilde{\sigma}_t = \mathrm{SNR}_t^{-1/2}$; for VP, $\tilde{\sigma}_t^2 = (1-\bar{\alpha}_t)/\bar{\alpha}_t$.

- Networks transfer between parametrizations after rescaling input and output; ODE paths agree up to α_t.
- The continuous-time ELBO depends only on the SNR at the two ends (Kingma et al. 2021).
- What differs is numerics: range and spacing of noise levels, network "preconditioning".

	α_t	σ_t	used in
VP	$\sqrt{\bar{\alpha}_t}$	$\sqrt{1-\bar{\alpha}_t}$	DDPM, Imagen, Stable Diffusion 1–2 (2020–22)
VE	1	$\sigma(t)$, e.g. $[0.002, 80]$	score SDE; EDM, consistency models
rectified flow	$1-t$	t	flow matching; Stable Diffusion 3, recent video models

VP gives the cleanest ELBO and ε-loss (our derivation). VE lives on through EDM. Current large models mostly use the straight path of **flow matching** $\Rightarrow$ a later lecture.

Summary

level	object	role
particles	forward chain, reverse SDE, DDPM sampler	what is simulated
densities	Fokker–Planck and its transport form	derives the reverse drift; probability-flow ODE
expectations	$\mathbb{E}[\varepsilon \mid x_t]$, $\mathbb{E}[x_0 \mid x_t]$ (Tweedie)	what the network learns = the score

1. Forward: an exact Gaussian chain shrinks data into $\mathcal{N}(0, I)$; any noise level in one shot.
2. Reverse: same noise, drift corrected by the score; Gaussian steps for small Δt with fixed variance.
3. Training: the ELBO reduces to predicting the added noise; the optimal predictor is the score (Tweedie).
4. Sampling: Euler–Maruyama for the reverse SDE; errors from start, score, and step size — not removed by MH.

Next: faster sampling (DDIM, ODE solvers), guidance and conditional generation, optimal transport and flow matching.

References

- J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *ICML*, 2015.
- J. Ho, A. Jain, P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- Y. Song, S. Ermon. Generative modeling by estimating gradients of the data distribution. *NeurIPS*, 2019.
- Y. Song et al. Score-based generative modeling through stochastic differential equations. *ICLR*, 2021.
- B. D. O. Anderson. Reverse-time diffusion equation models. *Stoch. Proc. Appl.*, 1982.
- P. Vincent. A connection between score matching and denoising autoencoders. *Neural Comput.*, 2011.
- A. Nichol, P. Dhariwal. Improved denoising diffusion probabilistic models. *ICML*, 2021.
- D. P. Kingma, T. Salimans, B. Poole, J. Ho. Variational diffusion models. *NeurIPS*, 2021.
- T. Karras, M. Aittala, T. Aila, S. Laine. Elucidating the design space of diffusion-based generative models. *NeurIPS*, 2022.
- S. Chen et al. Sampling is as easy as learning the score. *ICLR*, 2023.
- N. Carlini et al. Extracting training data from diffusion models. *USENIX Security*, 2023.